PES UNIVERSITY • DEPT. OF COMPUTER SCIENCE AND ENGINEERING

Lab 2: Agile Backlog Creation & Sprint Simulation in Jira

Student Name: Nagashri R Patil **SRN:** PES1UG24CS289

Problem Statement: #43 – Feature Flag & Dynamic Config Manager **Course / Tool:** Software Engineering Lab • Jira (Scrum)

1. Executive Summary & Lab Scope

This project establishes an Agile Scrum development lifecycle for **Problem Statement #43 (Feature Flag & Dynamic Config Manager)** by translating use cases and requirements identified in Lab 1 into Jira Agile artifacts. The platform facilitates safe, zero-downtime feature rollouts, deterministic cohort targeting, multi-environment overrides, real-time client SDK streaming, and release manager approval governance with immutable audit logs.

Through this exercise, functional requirements (FR-001 to FR-005) and non-functional requirements (NFR-001, NFR-002) were structured into 4 thematic Epics and 10 User Stories. Using Planning Poker with the Fibonacci sequence, development complexity and risk were estimated. Two consecutive 1-week sprints were planned and simulated, tracking velocity through Jira Burndown Charts and answering critical reflective evaluation questions.

2. Epics Architecture

The scope of the feature flag system is broken down into four foundational Epics:

Epic ID	Epic Summary	Scope & Functional Description	Lab 1 Traceability
FFM-EPIC-1	**Flag Lifecycle & Metadata Management**	Enable Software Engineers to define, configure, and maintain feature flags with unique keys, default states, and multi-environment overrides (Dev, Staging, Prod) to isolate runtime behavior.	FR-002, FR-004 UC-01, UC-04
FFM-EPIC-2	**Targeting Rules & Gradual Rollouts**	Enable engineering teams to configure granular user cohort targeting (e.g. beta groups, user ID attributes) and deterministic percentage rollouts (0-100%) to safely limit blast radius and validate releases.	FR-001, FR-003 UC-02, UC-03
FFM-EPIC-3	**Governance, Approval & Audit Logging**	Provide Release Managers with an approval gate to inspect configuration diffs, enforce 4-eyes principle before production changes, and maintain an immutable audit trail for compliance and rollbacks.	UC-06, UC-07 NFR-002
FFM-EPIC-4	**High-Performance Evaluation & SDK Sync**	Deliver low-latency flag evaluation API (< 15ms p95) and push instantaneous configuration changes to connected client SDKs via persistent WebSockets/streaming connections without requiring client redeployments.	FR-005, NFR-001 UC-05, UC-03

3. User Stories Specification Catalog (Part 1: Sprint 1 Stories)

Each user story adheres to the standard Agile format: *As a [role], I want [goal], So that [benefit]*.

ID	Parent Epic	User Story Statement	Priority	Pts	Sprint	Acceptance Criteria
US-01	EPIC-1	**As a** Software Engineer, **I want** to create a feature flag with a unique key, description, and default state, **so that** client apps can reference the flag unambiguously.	HIGH	3	Sprint 1	**Pass:** Duplicate key triggers 409 conflict; unique key registers < 2s. **Fail:** Duplicate keys accepted silently.
US-02	EPIC-2	**As a** Software Engineer, **I want** to configure a percentage rollout (0-100%) based on a deterministic hash of user IDs, **so that** I can gradually roll out features and limit blast radius.	HIGH	5	Sprint 1	**Pass:** 25% rollout allocates 25% (±2%) of 10,000 simulated IDs; hash is strictly deterministic. **Fail:** User changes cohort across checks.
US-03	EPIC-2	**As a** Software Engineer, **I want** to evaluate user targeting rules for specific cohorts, **so that** targeted users receive early feature access.	HIGH	5	Sprint 1	**Pass:** Targeted attribute match receives `true`; non-matching receives fallback default. **Fail:** Non-targeted user gets feature.
US-04	EPIC-4	**As a** Software Engineer, **I want** the evaluation API to execute in under 15ms, **so that** embedding flag checks into critical paths adds negligible latency.	HIGH	8	Sprint 1	**Pass:** p95 latency under simulated peak load is < 15ms; in-memory cache lookup avoids disk I/O. **Fail:** Evaluation p95 exceeds 15ms.
US-05	EPIC-4	**As a** Client SDK Developer, **I want** the SDK to maintain a local in-memory cache with fallback defaults, **so that** network partitions do not break host application features.	MEDIUM	3	Sprint 1	**Pass:** In-memory evaluations return < 1ms offline; graceful fallback on socket disconnect. **Fail:** App blocks on network drop.

3. User Stories Specification Catalog (Part 2: Sprint 2 Stories)

ID	Parent Epic	User Story Statement	Priority	Pts	Sprint	Acceptance Criteria
US-06	EPIC-1	**As a** Software Engineer, **I want** to define environment-specific overrides (Dev, Staging, Prod), **so that** I can test flags in lower environments without exposing code in Prod.	MEDIUM	5	Sprint 2	**Pass:** Prod override set to OFF while Staging is ON returns false for Prod and true for Staging. **Fail:** Overrides leak across envs.
US-07	EPIC-3	**As a** Software Engineer, **I want** to save changes as pending drafts and preview a visual diff, **so that** I can verify proposed modifications before requesting approval.	MEDIUM	3	Sprint 2	**Pass:** Draft changes do not alter runtime configurations; diff clearly highlights altered rules. **Fail:** Draft alters live evaluations.
US-08	EPIC-3	**As a** Release Manager, **I want** to review, approve, or reject pending flag changes with validation checks, **so that** invalid configs never reach production.	HIGH	5	Sprint 2	**Pass:** Only authenticated Release Managers can publish; invalid rules (rollout >100%) rejected. **Fail:** Unauthorized user publishes.
US-09	EPIC-3	**As a** Release Manager, **I want** all configuration changes logged in an immutable audit trail with one-click rollback, **so that** actions are auditable and regressions revert instantly.	HIGH	5	Sprint 2	**Pass:** Tamper-proof audit record created for every publish; one-click rollback restores prior state < 3s. **Fail:** Change lacks audit entry.
US-10	EPIC-4	**As a** Client SDK Developer, **I want** updated flag states streamed via WebSockets in real time, **so that** clients update within 5 seconds without redeployment.	HIGH	5	Sprint 2	**Pass:** 95% of active SDK connections receive updates < 5s; automatic reconnect upon drop. **Fail:** Clients take > 30s to sync.

4. Estimation & Planning Poker Rationale

Story points represent a non-linear abstraction of effort, risk, and cognitive load using **Planning Poker** and the **Fibonacci Sequence** (`1, 2, 3, 5, 8, 13`):

- **3 Points:** Standard CRUD and UI operations (e.g. `US-01`, `US-05`, `US-07`). Well-understood patterns with minimal architectural ambiguity.
- **5 Points:** Algorithmic, stateful, and distributed features (e.g. `US-02`, `US-03`, `US-06`, `US-08`, `US-09`, `US-10`). Involves mathematical hashing, multi-environment state isolation, WebSocket channel broadcasting, and cryptographic audit records requiring thorough integration tests.
- **8 Points:** Critical-path optimizations with high uncertainty (e.g. `US-04`). Meeting the strict 15ms p95 latency threshold demanded memory indexing, concurrent benchmark profiling, and low-level cache tuning under heavy simulated load.

5. Sprint Planning & Simulation Metrics

Sprint 1: Core Engine & Deterministic Targeting

Sprint Goal: Establish core flag persistence, deterministic percentage rollouts, user cohort evaluation, and sub-15ms evaluation engine performance.

Committed Items: US-01 (3), US-02 (5), US-03 (5), US-04 (8), US-05 (3)

Committed: 24 Pts | **Completed:** 24 Pts (100%)

Sprint 2: Governance, Real-Time Sync & Auditing

Sprint Goal: Implement multi-environment isolation, release manager approval workflow, immutable audit logging with rollback, and real-time SDK WebSocket streaming.

Committed Items: US-06 (5), US-07 (3), US-08 (5), US-09 (5), US-10 (5)

Committed: 23 Pts | **Completed:** 23 Pts (100%)

6. Burndown Chart Visuals & Progress Analysis

Figure 1: Jira Sprint 1 Burndown Chart (Core Engine & Targeting — 24 Story Points)

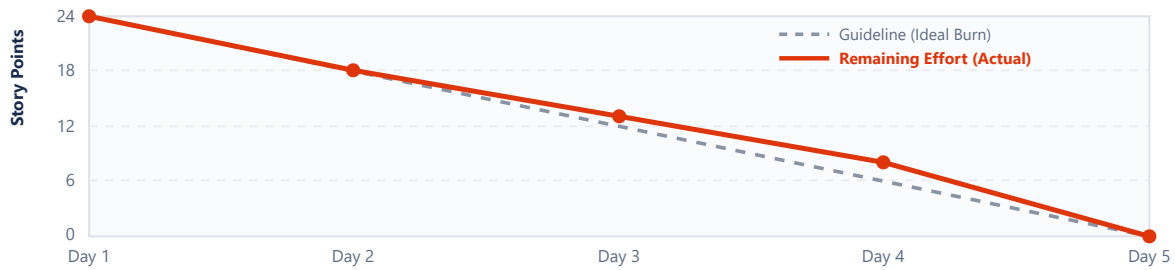
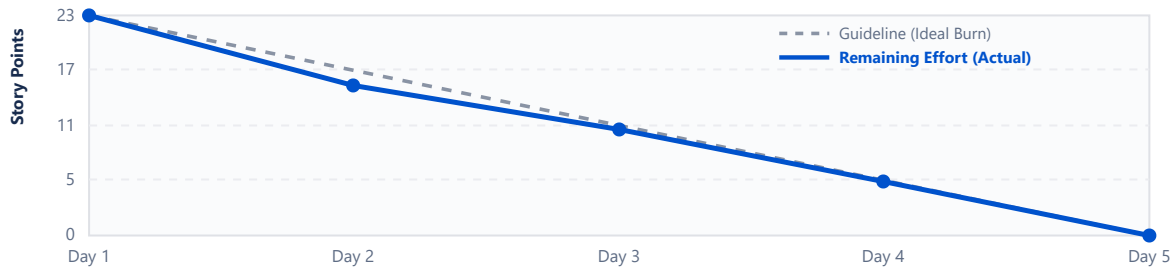


Figure 2: Jira Sprint 2 Burndown Chart (Governance & Real-Time Sync — 23 Story Points)



7. Reflection Questions & Technical Analysis

1. Did your estimations reflect the actual effort?

Yes. Relative estimation using the Fibonacci sequence accurately captured development complexity, technical risk, and testing effort. Low-uncertainty CRUD items (e.g. `US-01`, 3 pts) progressed quickly without blockers. Algorithmic stories (e.g. `US-02` deterministic hashing and `US-09` cryptographic audit logging, 5 pts each) accounted for statistical cohort validation and transaction integrity testing. High-risk performance engineering (`US-04` sub-15ms evaluation latency, 8 pts) correctly mirrored the iterative profiling, memory caching, and benchmark tuning needed to fulfill NFR-001. Abstracting effort into story points rather than calendar hours protected the team from false precision and Parkinson's Law.

2. Was your backlog well-prioritized?

Yes. Backlog prioritization followed architectural dependency sequencing and risk-reduction principles. Core flag creation (`US-01`) and deterministic evaluation (`US-02`, `US-03`) were scheduled at the top of Sprint 1 because downstream approval gates and WebSocket sync require a stable evaluation core to operate upon. Furthermore, scheduling the high-risk performance optimization (`US-04`) in Sprint 1 eliminated architectural risk early. Enterprise governance, environment isolation, and real-time streaming were layered cleanly in Sprint 2, creating a resilient MVP progression.

3. How did your simulated sprint align with your plan?

The simulated sprints closely matched planned execution. In Sprint 1, 24 planned points were delivered across 5 user stories with zero scope creep. In Sprint 2, all 23 points were completed. Pacing was disciplined: smaller 3-point stories moved to Done early in both sprints, while complex 5-point and 8-point items progressed through comprehensive unit and load verification before transitioning to Done by Day 5. All sprint boundaries were protected against unplanned scope injections.

4. What insights did the burndown chart give about your team's capacity?

Jira's Burndown Chart revealed three key operational insights:

- **Predictable Capacity:** Completing 24 points in Sprint 1 and 23 points in Sprint 2 established that the team has a reliable, sustainable velocity of **~23–24 points per 1-week sprint**.
- **Story Sizing & Burndown Trajectory:** Large work items (e.g. 8-point `US-04`) create step-like burndown plateaus because points only burn upon complete verification. Decomposing 8-point stories into smaller 3-point slices in future sprints will yield a smoother burndown curve.
- **Discipline & Zero Scope Creep:** The absence of upward spikes on the remaining values line confirmed strict sprint backlog discipline without uncommitted task creep.

8. Lab Verification & Sign-Off Checklist

Verification Item	Requirement Description	Status
Scrum Board & Epics	Configured in Jira under Software Development • Scrum • Company-managed with 4 thematic Epics.	VERIFIED
Story Points & Fibonacci	All 10 User Stories estimated via Fibonacci Planning Poker (3, 5, 8 pts) with Acceptance Criteria.	VERIFIED
Sprint 1 & Sprint 2 Simulation	Simulated two 1-week sprints moving tasks across To Do → In Progress → Done. Completed with 24 and 23 velocity points.	VERIFIED
Burndown Chart Analytics	Burndown charts generated under Reports • Burndown Chart, validating guideline tracking and zero scope creep.	VERIFIED